Backup

If the Wi-Fi or a live demo dies, teach from these.

Who’s talking

Senior Technical Lead, Boh

Building the EKGO! platform

Solana Students Africa

Co-established, backed by the Solana Foundation

I ship with AI every day

And I review every line it writes that touches money

Why stablecoins, why here

#3

Nigeria's rank worldwide for crypto adoption

Chainalysis Global Crypto Adoption Index, Sep 2026

~43%

Share of Sub-Saharan Africa's crypto volume that was stablecoins

Chainalysis, 2024 Sub-Saharan Africa report

~$15.5B

Stablecoins on Solana today. USDC is $6.8B of it

DefiLlama, 19 Oct 2026

One Wi-Fi, one IP, one rate limit

100

requests per 10 s, **per IP**, on the public devnet RPC

40

per 10 s for any single method

Use the workshop RPC URL

Put it in `NEXT_PUBLIC_SOLANA_RPC_URL`. It's browser-visible, so it must be origin-restricted.

Your wallet doesn’t hold USDC



Bug 1: the browser decides it’s paid

```
// app/page.tsx, AI first draft (unedited)
const sig = await sendTransaction(tx, connection);
const latest = await connection.getLatestBlockhash();
await connection.confirmTransaction({ signature: sig, ...latest }, "confirmed");
setSignature(sig);
setStatus("paid");
```

There is no server. The kitchen never finds out. And `setStatus("paid")` is one line in DevTools.

Bug 2: the price lives in the browser

```
// Reviewed: the server decides, the client asks
export const MENU = {
  jollof: { name: 'Jollof rice + chicken', price: 2_500_000n },
  suya: { name: 'Suya, one stick', price: 1_000_000n },
} as const

// POST /api/orders { item: 'jollof' }  →  { id, amount: '2500000', mint, recipient }
```

Then the server checks the chain says the merchant got **at least** that amount.

Bug 3: which jollof is this?

AI first draft

Two customers each pay 2.50 USDC to the same wallet, ten seconds apart.

Which payment is whose?

reviewed fix

```
getAddMemoInstruction({ memo: order.memo }) // "mamapu
```

```
const memoOk = ixs.some(ix =>
  ix.program === 'spl-memo' && ix.parsed === memoFor(o
```

The same payment, twice

ORDER #A1	
signature	3xuC…mF5K
verify	paid

ORDER #B7	
signature	3xuC…mF5K
verify	?

Each signature pays for **one** order, once.

Memo check + a used-signature record.
In production: a `UNIQUE` column.

The whole check, in four rules

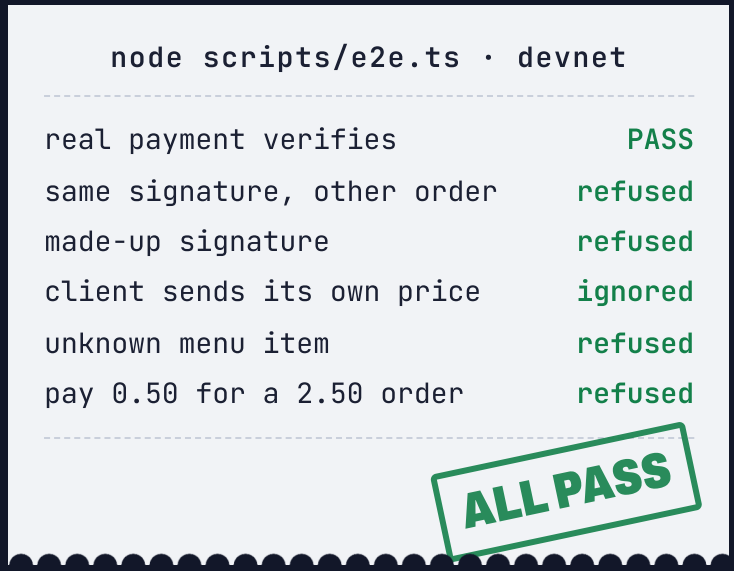
```
// app/lib/verify.ts
const tx = await rpc
  .getTransaction(sig, { commitment: 'confirmed', encoding: 'jsonParsed', maxSupportedTransactionVersion: 1 })
  .send()

if (tx.meta?.err) return fail('Transaction failed on-chain')

const memoOk = tx.transaction.message.instructions
  .some(ix ⇒ ix.program ≡≡ 'spl-memo' && ix.parsed ≡≡ memoFor(order))
if (!memoOk) return fail('Memo does not match this order')

// Merchant's balance of THIS mint must rise by the full price
const isMerchantUsdc = b ⇒ b.mint ≡≡ order.mint && b.owner ≡≡ order.recipient
const before = sum(pre.filter(isMerchantUsdc))
const after = sum(post.filter(isMerchantUsdc))
if (after - before < order.amount) return fail('Underpaid')
return ok()
```

Does it actually work? Attack it.



Real devnet transactions.

Same routes the browser uses.

Write the attack, then the fix.

Bug 4: confirming with the wrong blockhash

```
const sig = await sendTransaction(tx, connection);        // wallet adapter adds a blockhash here
const latest = await connection.getLatestBlockhash();      // ...then the AI fetches a NEWER one
await connection.confirmTransaction({ signature: sig, ...latest }, "confirmed");
```

It waits on a deadline the transaction doesn't have. If it's dropped, the page spins past the real expiry.

Fix: one client that builds, signs, sends and confirms with the same lifetime.

Bug 5: one token program assumed

AI first draft

```
getAssociatedTokenAddressSync(USDC_MINT, publicKey)
// defaults to the classic Token program
```

reviewed fix

```
const mint = await fetchEncodedAccount(rpc, USDC_MINT)
// mint.programAddress → Token or Token-2022
```

Fine for USDC. Wrong for **PYUSD** and **USDG**, which are Token-2022.

Bug 6: yesterday's stack

`AI first draft`

- `@solana/web3.js` v1
- `@solana/wallet-adapter-*`
- Next.js 15

`reviewed fix`

- `@solana/kit` 8
- `@solana/kit-plugin-wallet` + `@solana/react`
- Next.js 16

Not broken today. Debt from day one.

The Solana Foundation's dev skill for AI agents, since Jul 2026: don't use framework-kit or wallet-adapter for new work.

Prompt like a reviewer

You, second pass

Act as an attacker. List every way a customer could get this order marked paid without paying the full price to the right wallet. Then write a script that tries each one against my API, and fix the code until they all fail.

Better prompts help. They don't replace reading the code.

An agent wants to call your API

Sign up

needs an email and a human

Add a card

needs KYC and a billing address

Monthly plan

for one call worth $0.001

API key

one more secret to leak

What if the request carried the payment?

The real 402 from our API


```
{
  "x402Version": 2,
  "error": "Payment required",
  "resource": { "url": "http://localhost:4021/proverb", "description": "One proverb, paid per request" },
  "accepts": [{
    "scheme": "exact",
    "network": "solana:EtWTRABZaYq6iMfeYKouRu166VU2xqa1",
    "amount": "1000",
    "asset": "4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU",
    "payTo": "BzMDFqQe2D1QnXkUiponJCasNiCZn8zEtW4VVehXLikf",
    "maxTimeoutSeconds": 300,
    "extra": {
      "feePayer": "CKPKJWNdJEqa81x7CkZ14BVPiY6y16Sxs7owznqtWYp5"
    }
  }]
}
```

The server: one middleware

```
// x402/src/server.ts
import { paymentMiddleware, x402ResourceServer } from '@x402/express'
import { HTTPFacilitatorClient } from '@x402/core/server'
import { ExactSvmScheme } from '@x402/svm/exact/server'

const SOLANA_DEVNET = 'solana:EtWTRABZaYq6iMfeYKouRu166VU2xqa1'

const facilitator = new HTTPFacilitatorClient({ url: 'https://x402.org/facilitator' })
const resourceServer = new x402ResourceServer(facilitator).register(SOLANA_DEVNET, new ExactSvmScheme())

app.use(paymentMiddleware({
  'GET /proverb': {
    accepts: [{ scheme: 'exact', price: '$0.001', network: SOLANA_DEVNET, payTo: PAY_TO }],
    description: 'One proverb, paid per request',
  },
}, resourceServer))

app.get('/proverb', (_req, res) => res.json({ proverb: pick(PROVERBS) }))
```

The agent: wrap fetch

```
const client = new x402Client()
client.register('solana:*', new ExactSvmScheme(signer))

client.setSpendControls({ maxAmountPerPayment: '$0.01' })
const paidFetch = wrapFetchWithPayment(fetch, client)

const res = await paidFetch('http://localhost:4021/proverb')
console.log(await res.json()) // { proverb: 'Rain does not fall on one roof alone.' }
```

The budget line matters. **An agent with a wallet and no limit is a liability.**

Trust the facilitator, carefully

15 of 15

real-world x402 facilitators tested broke at least one of eight security rules

"When HTTP 402 Meets the Blockchain: Risks on Emerging x402 Payments" · arXiv 2607.19545 · USENIX Security 2026

Pin the SDK version. Fail closed if the facilitator errors. Settle before you hand over anything valuable.

Two speeds, one ecosystem

Checkout app

`@solana/kit` 8.4

x402 service

`@x402/svm` 2.28 → `@solana/kit` 5

Same project?

`npm ERR! ERESOLVE could not resolve`

Our call

Two small services. Don't force it with `--legacy-peer-deps`.

Who’s behind x402

Apr 2026

Linux Foundation launches the x402 Foundation. Initially developed by Coinbase, Cloudflare and Stripe.

Jul 2026

40 members. Premier: Solana Foundation, Visa, Google, Mastercard, AWS, Stripe, Circle and more.

76%

of x402 transactions run on Solana, 22 Aug to 19 Sep 2026: 23.2M. Solana's figure, citing Artemis.

Where this is going

Payments are turning into **an HTTP header**.

Customers are turning into **programs with wallets**.

Writing the code is getting cheap. **Knowing it's right isn't.**